PAYRECON

Project Explained

How a payment ledger and bank reconciliation system works, explained in plain English, with pictures.

Payment ledger + bank reconciliation · Java 17 · MySQL · Hadoop MapReduce · Hive · MongoDB

At a glance

What it is	A working miniature of the ledger and reconciliation systems that payment companies such as Stripe and Razorpay run
Built with	Java 17, MySQL 8.4, Hadoop MapReduce 3.3, Hive 4.0, MongoDB 7.0, Docker
Test month	1,000 shops, 2,00,000 payments (₹25.1 crore), 105 bank files in three formats
Results	99.83% matched automatically; every planted problem found; every bank ties out to the paisa

This document explains the whole project in simple English. When a technical word appears for the first time it is in **bold**, and its meaning comes right after it. All the words are collected again in the glossary at the end.

Contents

1. The problem, as a story 3
2. The big picture 4
3. Part 1: the ledger (MySQL) 4
3.1 What a ledger is, and why "double-entry" . . . 4
3.2 Accounts and their "normal side" . . . 5
3.3 Worked example: Riya's ₹1,000 payment . . . 5
3.4 Why money is stored in paise, never as a decimal number . . . 6
3.5 The MySQL tables (15) . . . 6
3.6 Proving the ledger is right: ./payrecon audit . . . 7
4. Part 2: the REST API, idempotency and concurrency 7
4.1 The API . . . 7
4.2 Idempotency: never charge twice . . . 7
4.3 Race conditions: the refund problem . . . 8
4.4 Deadlocks and lock order . . . 9
4.5 The stress test: ./payrecon stress . . . 9
5. Part 3: the bank files and the data generator 10
5.1 Three banks, three formats . . . 10
5.2 The generator and the answer key . . . 10
6. Part 4: matching with Hadoop MapReduce 11
6.1 What MapReduce is . . . 11
6.2 Job 1: exact reference (a reduce-side join) . . . 12
6.3 Job 2: amount + date window (secondary sort) . . . 12
6.4 Job 3: payout batches (a combiner) . . . 13
6.5 Running the jobs as a DAG . . . 14
6.6 Why MapReduce here? . . . 14
7. Part 5: after matching (exceptions, cash application, tie-out) 14
7.1 Exceptions go to MySQL . . . 14
7.2 Cash application, and a real bug . . . 14
7.3 The tie-out: the accountant's proof . . . 15
8. Part 6: Hive (the analytics warehouse) 16
9. Part 7: MongoDB (documents) 17
10. Part 8: the exceptions workflow (MySQL) 18
11. Part 9: how we know it is correct 19
12. Results (October 2026: 1,000 shops, 200,000 payments, MacBook, Docker with 4 GB) 20
13. Design decisions, trade-offs and limits 20
14. Where things are in the code 21
Glossary 21

1. The problem, as a story

Riya buys shoes for ₹1,000 from *Sharma Footwear*'s website and pays with UPI on 10 October.

- *Sharma Footwear* does not deal with banks directly. It uses **PayRecon**, a **payment aggregator**: a company that accepts online payments on behalf of many shops. Stripe, Razorpay and PayU are real ones.
- Riya's money does not reach PayRecon instantly. The **acquiring bank** (the bank that processes card and UPI payments for PayRecon; here HDFC) collects the day's payments and sends the money the next day, all together. This transfer is called a **settlement**. Along with the money, the bank sends a **settlement file**: a list of every payment included in that transfer.
- PayRecon keeps a fee (say 2% = ₹20) and later sends the remaining ₹980 to Sharma Footwear. That transfer is a **payout**.

Now multiply this by thousands of payments a day, three banks, and hundreds of shops. Things go wrong all the time:

- the bank never settles some payments
- the bank pays ₹999.99 instead of ₹1,000
- the bank pays the same payment twice
- the bank cuts the payment reference short, or drops it
- a line in the bank's file is broken and cannot be read
- the bank takes an extra ₹25 charge when doing our payouts, or never does a payout at all

Reconciliation means comparing our own records with the bank's records, line by line, and explaining every difference. Every payment company has systems and teams just for this, because each unexplained difference is real money lost. PayRecon is a working miniature of such a system.

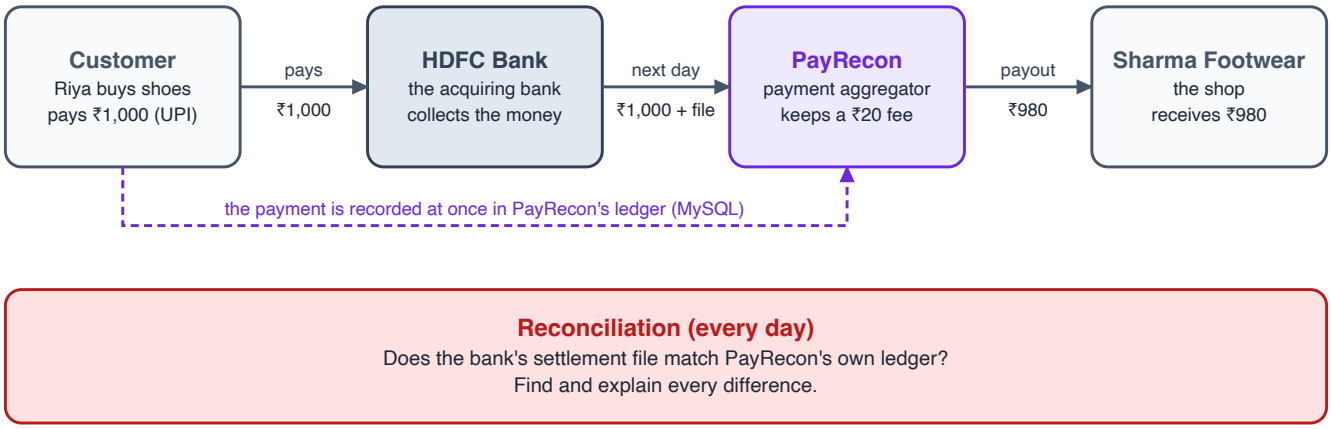


Figure 1. How money moves for one ₹1,000 payment, and where reconciliation fits in.

2. The big picture

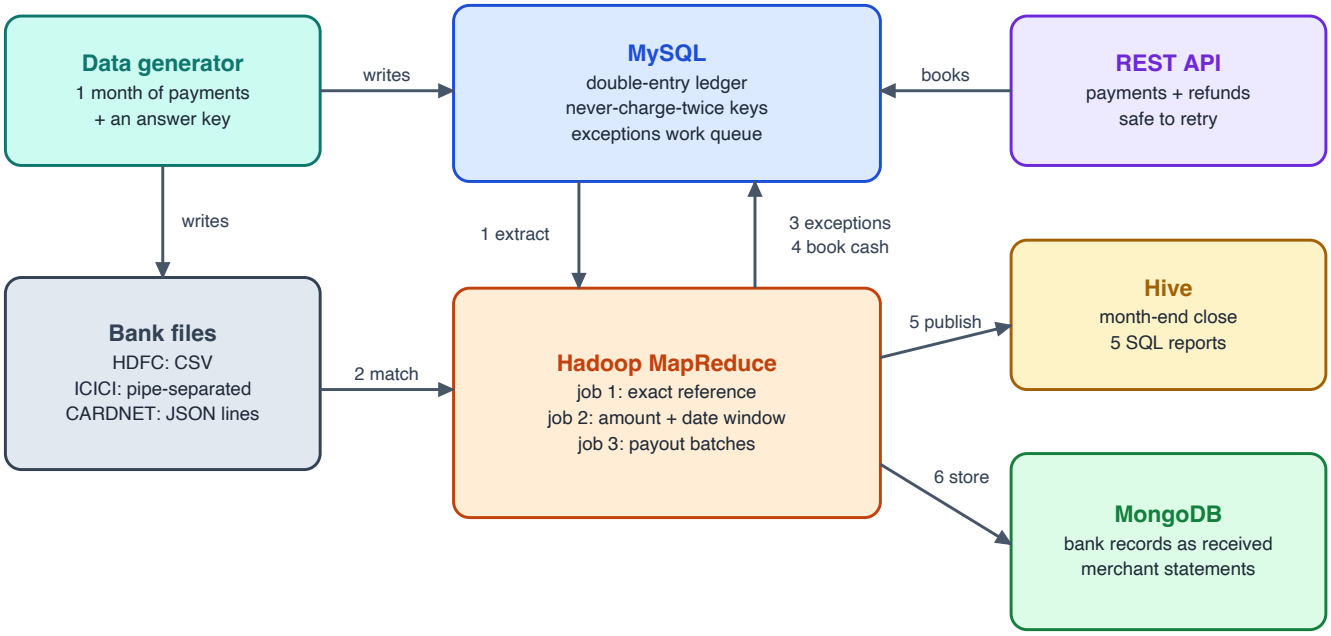


Figure 2. The big picture. The numbers 1 to 6 are the six steps of every reconciliation run.

Each of the four technologies has a job that fits what it is good at:

Technology	Job in PayRecon	Why it fits
MySQL	the ledger, idempotency keys, exception queue	money needs transactions, locks and constraints
Hadoop MapReduce	the matching engine	custom matching logic over every record, split across workers
Hive	month-end close and analytics	SQL over the full history of files, away from the live database
MongoDB	raw bank records, merchant statements	every bank's format is different; documents keep each shape as-is

3. Part 1: the ledger (MySQL)

3.1 What a ledger is, and why "double-entry"

A **ledger** is the official record of every money movement in a business.

Double-entry bookkeeping is a 700-year-old accounting method. Every movement is written down at least twice: money always comes **from** somewhere and goes **to** somewhere. The two sides are called **debit** and **credit**. The one rule:

> *In every **journal entry** (the record of one business event, such as "payment captured"), total debits = total credits.*

Each line of a journal entry is a **posting**: an account, a direction (debit or credit) and an amount.

Why bother? Because if every entry balances, the whole book balances, so a single SQL query can detect a bug anywhere in the system. A ledger that only stores "balance = 1,500" cannot tell you whether that 1,500 is right. A double-entry ledger can.

3.2 Accounts and their "normal side"

An **account** is a named bucket of money. PayRecon's **chart of accounts** (the list of all accounts) is:

Account	Type	Meaning
`CLEARING:HDFC`, `CLEARING:ICICI`, `CLEARING:CARDNET`	asset	money each bank owes us for payments it has not settled yet
`SETTLEMENT_CASH`	asset	cash in our own bank account
`MERCHANT_PAYABLE:<shop>`	liability	money we owe each shop
`REFUNDS_PAYABLE`	liability	money we owe customers for refunds
`SUSPENSE`	liability	money we received but cannot identify yet
`FEE_REVENUE`	revenue	our fees

- An **asset** is something we own or are owed.
- A **liability** is something we owe.
- **Revenue** is what we earned.

Each type has a **normal side**: assets grow when debited, while liabilities and revenue grow when credited. It is only a convention. PayRecon stores every balance with its normal side as positive.

3.3 Worked example: Riya's ₹1,000 payment

Payment captured (`LedgerService.paymentLegs`):

Account	Debit	Credit
CLEARING:HDFC (HDFC now owes us)	1,000	
MERCHANT_PAYABLE:Sharma (we owe the shop)		980
FEE_REVENUE (we earned)		20
Total	**1,000**	**1,000**

Next day, HDFC settles. Once reconciliation has matched the money to the payment, cash application books it:

Account	Debit	Credit
SETTLEMENT_CASH (cash arrived)	1,000	
CLEARING:HDFC (HDFC no longer owes it)		1,000

Payout to the shop:

Account	Debit	Credit
MERCHANT_PAYABLE:Sharma	980	
SETTLEMENT_CASH (cash leaves)		980

Account (type)	After the payment	After the bank settles	After the payout
CLEARING:HDFC (asset)	₹1,000	₹0	₹0
SETTLEMENT_CASH (asset)	₹0	₹1,000	₹20
MERCHANT_PAYABLE (liability)	₹980	₹980	₹0
FEE_REVENUE (revenue)	₹20	₹20	₹20
Check: assets = liabilities + revenue	1,000 = 980 + 20	1,000 = 980 + 20	20 = 0 + 20

Figure 3. Each account's balance after each step of Riya's payment. The accounting equation (last row) holds after every step: that is the point of double-entry.

Refund of ₹200: debit `MERCHANT_PAYABLE` 200, credit `REFUNDS_PAYABLE` 200. The shop is owed less, and the customer is owed the refund.

Money we cannot identify: debit `SETTLEMENT_CASH`, credit `SUSPENSE`. A **suspense account** is a parking place for money whose purpose is not yet known. Accountants must eventually empty it ("clear the suspense") by finding out what each amount was for.

3.4 Why money is stored in paise, never as a decimal number

Computers store `double` and `float` numbers in binary, and most decimal fractions cannot be stored exactly. In Java, `0.1 + 0.2` gives `0.30000000000000004`. Repeated over millions of payments, those tiny errors become real missing paise.

So PayRecon stores every amount as a whole number of **paise** in a `long` (₹1,234.50 = `123450`). Rupee text from bank files is converted with `BigDecimal`, which refuses anything with more than 2 decimals (`Money.parseRupees`). Fees use integer arithmetic too:

```
fee = (amount × bps + 5000) / 10000
```

Here **bps** means basis points (1 bps = 0.01%), and adding 5000 before dividing rounds half-up.

3.5 The MySQL tables (15)

- **Ledger:**
  - `merchant` and `account`
  - `journal_entry` and `posting` (the double-entry core)
  - `payment`, `refund`, `payout_batch` and `payout` (the business records, each linked to its journal entry)
- **API safety:** `idempotency_key` (section 4).
- **Reconciliation:**
  - `recon_run` (one row per run, with counts and runtime)
  - `recon_pass_stat` and `recon_step_timing`
  - `cash_application` (section 7)
  - `recon_exception` (the work queue) and `exception_event` (the audit trail)

The tables protect themselves with **constraints**: rules the database enforces no matter which program writes to it.

- Every posting amount must be positive: `CHECK (amount_minor > 0)`.
- A payment can never be refunded more than it was paid: `CHECK (refunded_minor <= amount_minor)`.
- Every posting must point to a real account and a real entry. A **foreign key** is a column that must match a row in another table.
- One business event can only be booked once: a **unique key** on `(entry_type, external_ref)`.

3.6 Proving the ledger is right: `./payrecon audit`

Nine checks, each one SQL query:

1. Every journal entry balances.
2. Every entry has at least two postings.
3. **Trial balance:** the sum of all debits equals the sum of all credits.
4. Each stored account balance equals the sum of its postings.
5. **Accounting equation:** assets = liabilities + revenue − expenses.
6. No payment is refunded more than it was paid.
7. Each payment's refunded total equals the sum of its refunds.
8. The cash booked to the clearing accounts equals the per-line cash record (section 7).
9. The suspense account equals the unidentified money on record.

After 200,000 payments and a stress test, all nine pass.

4. Part 2: the REST API, idempotency and concurrency

4.1 The API

An **API** (application programming interface) lets other programs use PayRecon. Ours is a **REST API**: programs send **HTTP requests** to URLs called **endpoints**, for example `POST /v1/payments` to create a payment. Every answer carries an **HTTP status code**:

Code	Meaning
200	OK
201	created
400	your request is wrong
404	not found
409	conflict with the current state
422	the request cannot be processed

It runs on the JDK's built-in `HttpServer`, with no web framework, so every line is visible (`api/ApiServer.java`).

4.2 Idempotency: never charge twice

Riya's phone sends "pay ₹499". The network is slow, the app times out, and it sends the request again. Without protection, Riya pays twice.

An operation is **idempotent** if doing it twice has the same effect as doing it once. An **idempotency key** is a unique id the client creates for one intended operation and sends in a header (`Idempotency-Key: ...`). If the client retries, it sends the same key.

How PayRecon makes `POST /v1/payments` idempotent, all inside **one database transaction**:

1. Insert the key, plus a fingerprint (SHA-256 hash) of the request, into `idempotency_key`. The key is the table's **primary key**, so it is unique.
2. Book the payment: the journal entry, its postings, the balances, and the payment row.
3. Store the response next to the key.
4. Commit.

A **transaction** is a group of database changes that either all happen or none happen. Databases promise **ACID** for transactions:

- **Atomic:** all or nothing.
- **Consistent:** the rules always hold.
- **Isolated:** transactions running at the same time do not see each other's half-finished work.
- **Durable:** once committed, the change survives a crash.

What happens when the **same key arrives twice at the same moment**?

1. Request A inserts the key. That row is now **locked**: no one else may change it until A finishes.
2. Request B tries to insert the same key, so it **waits**.
3. A commits. B now gets "duplicate key", reads A's stored response and returns it with the header `Idempotent-Replayed: true`.

Exactly one payment is booked. If B used the same key with a *different* request body, it gets `422`. Because the key and the payment are saved in the same transaction, there is no moment where one exists without the other.

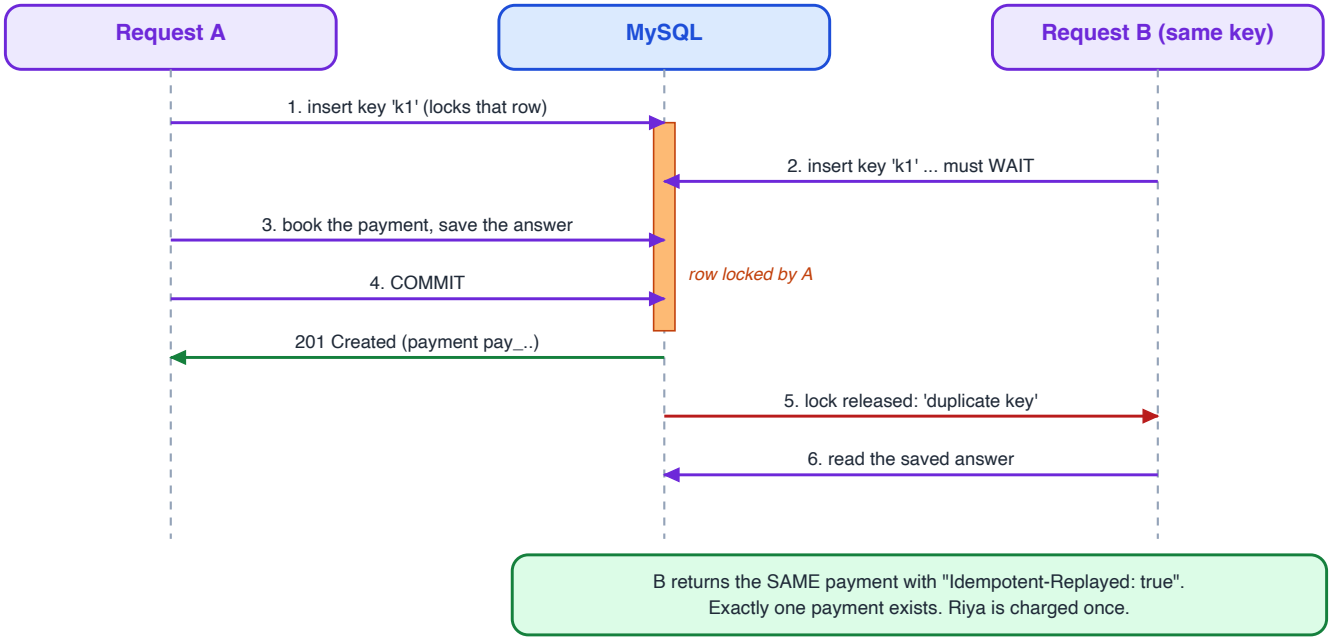


Figure 4. Two requests with the same Idempotency-Key arrive at the same moment. Only one payment is ever booked.

4.3 Race conditions: the refund problem

A **race condition** is a bug where the result depends on the timing of two things running at once.

- A ₹1,000 payment gets two refund requests of ₹600 at the same instant.
- Both read "refunded so far: ₹0", both check "600 ≤ 1,000: OK", and both refund.
- ₹1,200 has now been refunded on a ₹1,000 payment.

PayRecon's fix is `SELECT ... FROM payment WHERE payment_id = ? FOR UPDATE`. This reads the row *and locks it*, so the second refund waits until the first commits and then reads the updated total. Locking first and checking second is called **pessimistic locking**.

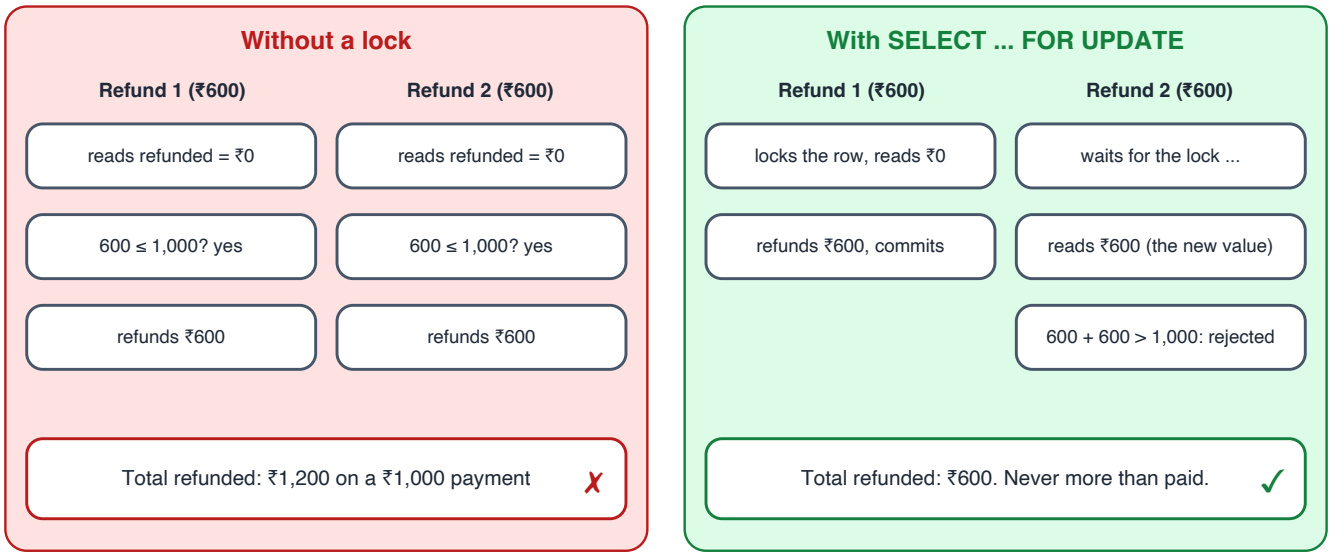


Figure 5. Two refunds of ₹600 on a ₹1,000 payment arrive together. Locking the row makes the second one wait.

On top of that, the database CHECK constraint (`refunded_minor <= amount_minor`) is a safety net. Even a buggy program cannot over-refund.

4.4 Deadlocks and lock order

A **deadlock** happens when transaction A holds lock 1 and waits for lock 2, while transaction B holds lock 2 and waits for lock 1. Neither can continue. The database notices, kills one of them, and reports error 1213.

PayRecon avoids deadlocks in `LedgerService.postEntry` with two rules:

1. **Always lock accounts in the same order** (sorted by `account_id`). If everyone takes locks in the same order, a circle of waiting cannot form.
2. **Update account balances before inserting postings.** Inserting a posting makes MySQL check its foreign key, which takes a *shared* lock on the account row. If we later update that account, the shared lock must become an *exclusive* lock. Two transactions doing that at the same time deadlock each other.

As a final safety net, `Db.inTransaction` retries a transaction up to 5 times if MySQL still reports a deadlock.

The **isolation level** is how much transactions running at the same time can see of each other. PayRecon uses **READ COMMITTED**: a transaction only sees data that other transactions have already committed. In MySQL it also takes fewer "gap" locks than the default level, which means fewer deadlocks.

4.5 The stress test: `./payrecon stress`

All threads wait behind a **start gate** (a Java `CountDownLatch`), then fire at exactly the same moment, so the races really happen.

Scenario	Result
50 requests at once, same idempotency key	1 created, 49 replayed, 1 row in MySQL
20 refunds of ₹100 at once on a ₹1,000 payment	10 succeed, 10 rejected, total exactly ₹1,000
200 different payments at once, 5 shops	all 200 created (about 220 per second on a laptop)
6 analysts claim exceptions at once	6 different items (section 10)

The ledger audit passes afterwards.

5. Part 3: the bank files and the data generator

5.1 Three banks, three formats

Real banks all send different formats. PayRecon simulates three:

HDFC: CSV (comma-separated values). Amounts are in rupees, and the reference is hidden inside a free-text narration:

```
value_date,utr,narration,credit_amount,debit_amount,currency
2026-10-11,UTR4416523301,SETTLEMENT/PAY REF pay_0000012345,1000.00,,INR
2026-10-12,UTR0193847756,UPI CREDIT RRN601234567890,500.00,,INR          <- no reference at all
2026-10-12,UTR5521093844,PAYOUT BATCH pob_20261012_HDFC,,1234567.89,INR  <- money going out
```

ICICI: pipe-separated fields. Amounts are in paise padded with zeros, and the reference field is sometimes cut to 12 characters:

```
CR|20261011|RRN601234567890|pay_0000012345|00000100000|INR
CR|20261011|RRN601234567891|pay_00000123|00000049900|INR               <- reference cut short
```

CARDNET: JSON lines. There is one JSON object per line, with nested objects inside:

```
{"settlement_id":"stl_2755758982","settled_on":"2026-10-12","txn":{"merchant_ref":"pay_0000012345",
  "auth_code":"360612"},"gross":{"value":"1000.00","currency":"INR"},"fees":{"interchange":"0.00",
  "scheme":"0.00"}}
```

Some words from these files:

- **Value date:** the day the bank moved the money.
- **UTR / RRN:** the bank's own transaction numbers.
- **Credit line:** money coming in.
- **Debit line:** money going out.

There is exactly **one parser** for all of this (`bank/BankParsers.java`). Both MapReduce and MongoDB use it, so the parsing rules cannot drift apart. A line it cannot read is never thrown away: it becomes a `MALFORMED_BANK_LINE` exception.

5.2 The generator and the answer key

`./payrecon generate` creates one month of business:

- shops of very different sizes (a few big ones, many small ones), with fees of 1.5–2.5%
- payments spread over the day, where 35% of UPI payments are round amounts like ₹500
- refunds
- daily and weekly payouts, grouped into **payout batches**: all payouts sent to one bank on one day travel as one bank debit

It writes the bank files and **plants problems** at small, realistic rates:

Planted problem	Rate	Count (200k payments)
bank never settles a payment	0.05%	92
bank pays a slightly different amount	0.1%	205
bank sends the same payment twice (next day's file)	0.05%	105
reference cut short (ICICI) or missing (HDFC UPI)	0.5% / 0.3%	585
money from the bank matching no payment	about 0.03%	60
unreadable line	about 0.01%	20

Planted problem	Rate	Count (200k payments)
bank debits a wrong payout batch total (+₹25 charge)	3% of batches	3
bank never executes a payout batch	2% of batches	2

Every planted problem goes into `data/answer_key.csv`, with the exact line it is on. This is the **ground truth**: the known right answer we can test against.

The generator uses a **seed**: a starting number for its random choices. The same seed always produces exactly the same data, so results are repeatable.

6. Part 4: matching with Hadoop MapReduce

6.1 What MapReduce is

Hadoop is an open-source system for processing big data on many computers. **MapReduce** is its programming model, with three phases:

1. **Map.** Read each input record and turn it into **key-value pairs**: a key (what to group by) and a value (the data).
2. **Shuffle and sort.** The framework collects all pairs with the same key, sorted by key, and sends each group to one worker.
3. **Reduce.** Process each key's group of values and write the result.

Think of an exam hall:

- Map: every invigilator stamps each answer sheet with its roll number.
- Shuffle: the sheets are sorted into piles, one pile per roll number.
- Reduce: one teacher marks each pile.

Because each pile is independent, you can add more teachers, which in Hadoop means more machines.

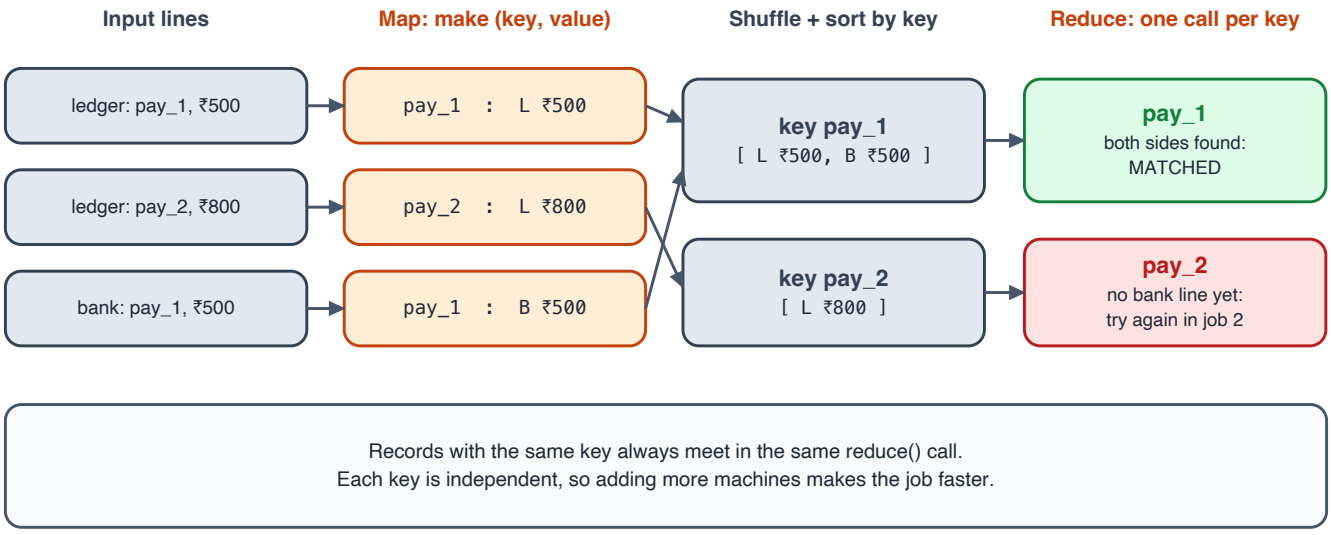


Figure 6. MapReduce in three steps, using job 1 as the example: a payment and its bank line meet by reference.

PayRecon runs Hadoop in **local mode**: the whole framework runs inside our Java program, with no cluster. The same job classes would run unchanged on a real cluster.

The engine is three **jobs** (a job is one complete map–shuffle–reduce program).

6.2 Job 1: exact reference (a reduce-side join)

A **join** combines records from two sources that share a key, here the payment reference `pay_0000012345`.

- The mapper reads **both** the ledger extract and the raw bank files. It **tags** each record with its source, `L` (ledger) or `B` (bank), and emits it with the reference as the key.
- After the shuffle, each reduce call receives one payment *and* every bank line with its reference, together. Doing the join in the reducer is called a **reduce-side join**.

The reducer (whose rules live in the pure function `ExactRefMatcher`) decides:

Situation	Result
1 payment, 1 bank line, same amount	matched
1 payment, 1 bank line, different amount	matched **and** `AMOUNT_MISMATCH` (with the difference)
1 payment, 2+ bank lines	first one matched, the others are `DUPLICATE_IN_BANK`
bank line whose reference is not in the ledger	`MISSING_IN_LEDGER`
payment with no bank line	sent to job 2

Other Hadoop features job 1 uses:

- **MultipleOutputs** lets one job write several output folders: `matched/`, `exceptions/`, `unmatchedledger/`, `pass2bank/` (bank lines with a short or missing reference), `payoutbank/` (debits), `malformed/`, and `bank_normalized/value_date=…/` for Hive.
- **Counters** are global tallies every worker can increase cheaply. PayRecon counts lines per bank and unreadable lines with them.
- **A non-splittable input format.** Hadoop normally cuts big files into pieces (**splits**) for different mappers. Each bank line's id includes its *physical line number*, and only a mapper that reads a file from the start can count lines. So PayRecon forces one file per mapper (`NonSplittableTextInputFormat`).

A real bug found while building: if you sort line ids as text, `file:10` comes before `file:9`, so the wrong copy of a duplicate was called "the original". Bank lines are now sorted by date, then file name, then line **number**, and a unit test guards it.

6.3 Job 2: amount + date window (secondary sort)

Some bank lines can't be matched by reference: ICICI cut it short, or the HDFC UPI line has none. A human would look for a payment with **the same bank, the exact same amount and a nearby date**. Job 2 does exactly that.

To make this efficient, it uses a classic MapReduce technique called **secondary sort**:

- The map output key is a **composite key** (a key made of several parts): `(bank|amount, date, kind, id)`.
- A custom **partitioner** (decides which reducer gets a record) looks only at `bank|amount`.
- A custom **grouping comparator** (decides which records share one `reduce()` call) also looks only at `bank|amount`.
- The full key is still used for **sorting**.

The result: each `reduce()` call receives *all* payments and bank lines for one bank and one amount, **already sorted by date**, without the reducer sorting anything in memory.

Inside, `WindowMatcher` walks the two date-sorted lists with a **two-pointer sliding window**. One pointer marks the earliest date still allowed, and it only moves forward. Each record is therefore looked at a small, fixed number of times: O(n + m), not O(n × m). The candidates for a bank line are unmatched payments from (bank date – normal lag – 4 days) up to the bank date. If the bank kept part of the reference, the payment id must also start with it.

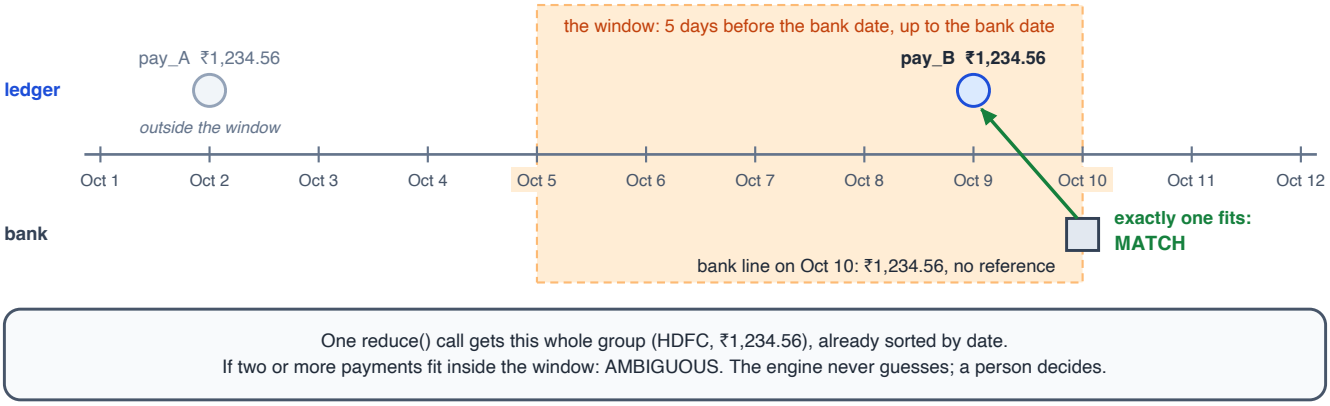


Figure 7. Job 2 for one group: same bank, same amount, and a date inside the window.

- **Exactly one candidate:** matched.
- **No candidate:** `MISSING_IN_LEDGER`.
- **Two or more candidates:** `AMBIGUOUS_MATCH`, and the candidate payments become `PENDING_REVIEW`. **The engine never guesses with money.** If two ₹500 UPI payments fit one ₹500 bank line, a person decides.

Payments still left over are:

- `IN_TRANSIT` if they are still young. The bank normally pays the next day (CARDNET after two), plus 2 days of grace. This is normal and not an error.
- `MISSING_IN_BANK` if they are older than that.

In the 200k run, job 2 matched 515 lines by amount and date. Of the 585 planted lines with a short or missing reference, 526 were matched automatically and 59 were sent to a person because several payments fit.

6.4 Job 3: payout batches (a combiner)

When PayRecon pays 31 shops on one day through HDFC, the bank shows **one** debit for the whole batch. This is a **many-to-one** match. Job 3 adds up the ledger's payouts per batch and compares the total with the bank debit:

- **Equal:** matched.
- **Different:** `PAYOUT_BATCH_MISMATCH`, for example when the bank took a ₹25 charge.
- **No debit found:** `PAYOUT_NOT_EXECUTED`, or `PAYOUT_PENDING` if the batch is dated on the cutoff day itself.
- **A debit matching no batch:** `UNKNOWN_DEBIT`.

A **combiner** is a mini-reducer that runs on the map side, before data travels through the shuffle. Instead of shipping hundreds of payout records per batch, each mapper ships one partial sum. This is only safe because addition is **associative**: (a+b)+c = a+(b+c), so partial sums can be added again later in any grouping.

6.5 Running the jobs as a DAG

A **DAG** (directed acyclic graph) is a set of steps with arrows showing what depends on what, with no loops. Here:

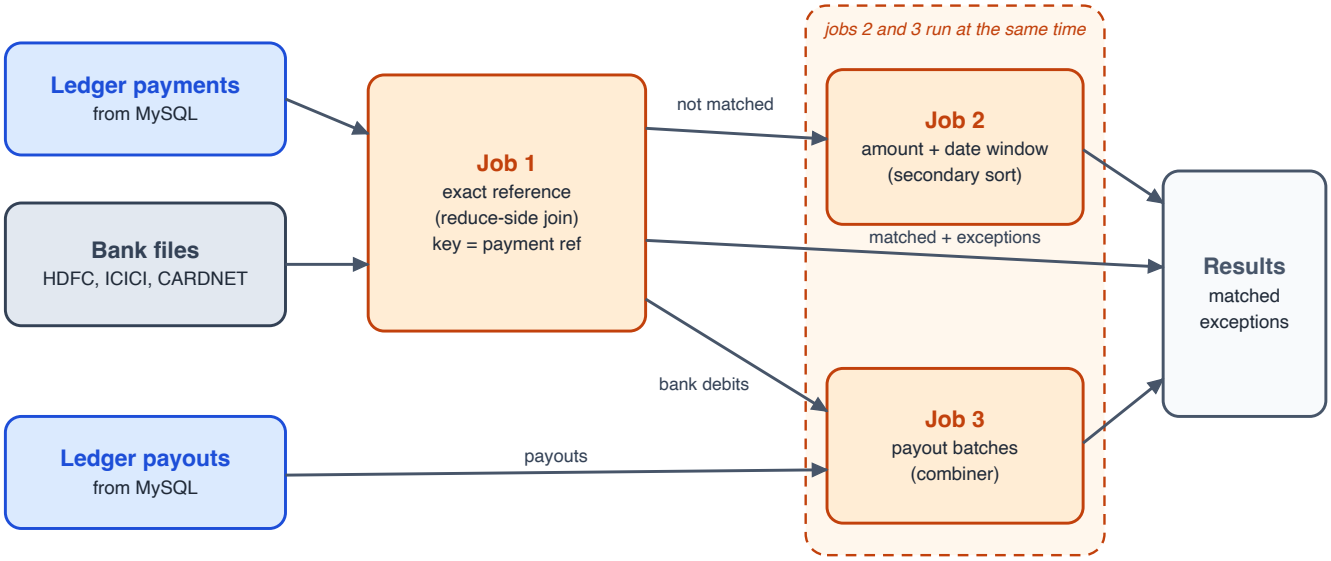


Figure 8. The three MapReduce jobs as a DAG: job 1 runs first, then jobs 2 and 3 run in parallel.

Jobs 2 and 3 both need only job 1's output, so PayRecon submits them together and they run in parallel (`ReconJobs.run`).

6.6 Why MapReduce here?

Matching needs **custom logic per group**: sliding windows, ambiguity rules and many-to-one sums. That is awkward in plain SQL but natural in a reducer. MapReduce also scales by adding machines, because each reference or bank+amount group is independent.

At 200,000 payments one laptop is enough. In a real company today this would usually run on **Spark** (a newer engine that keeps data in memory), but Spark is built on exactly these ideas: map, shuffle, partitioners and sort keys.

7. Part 5: after matching (exceptions, cash application, tie-out)

7.1 Exceptions go to MySQL

All exception rows are loaded into `recon_exception`, sorted so the **actionable** ones (needing a person) come first, biggest amount first. The informational ones (`IN_TRANSIT`, `PENDING_REVIEW`, `PAYOUT_PENDING`) are kept but not queued.

7.2 Cash application, and a real bug

Cash application means booking the money the banks *actually* paid into the ledger, once we know what it was for:

- **Matched credits:** debit `SETTLEMENT_CASH`, credit `CLEARING:<bank>`.
- **Unidentified credits** (no matching payment, duplicates, ambiguous lines): debit `SETTLEMENT_CASH`, credit `SUSPENSE`.

The first version booked one entry per (bank, day) and skipped days already booked. Testing found a real problem:

1. In run 1, two ₹1,000 bank lines were ambiguous, so their money went to suspense.

2. In run 2, with more bank files, they became uniquely matched.
3. But that day was "already booked", so the money stayed in suspense, and the tie-out showed a **₹2,000 difference**.

The fix is the `cash_application` table, which remembers **for every bank line** where its money was booked. Each run books only the *changes*:

Before → now	Ledger entry
new line	book it to clearing or suspense
suspense → matched	debit `SUSPENSE`, credit `CLEARING` ("clearing the suspense account")
matched → unidentified	the reverse
unchanged	nothing

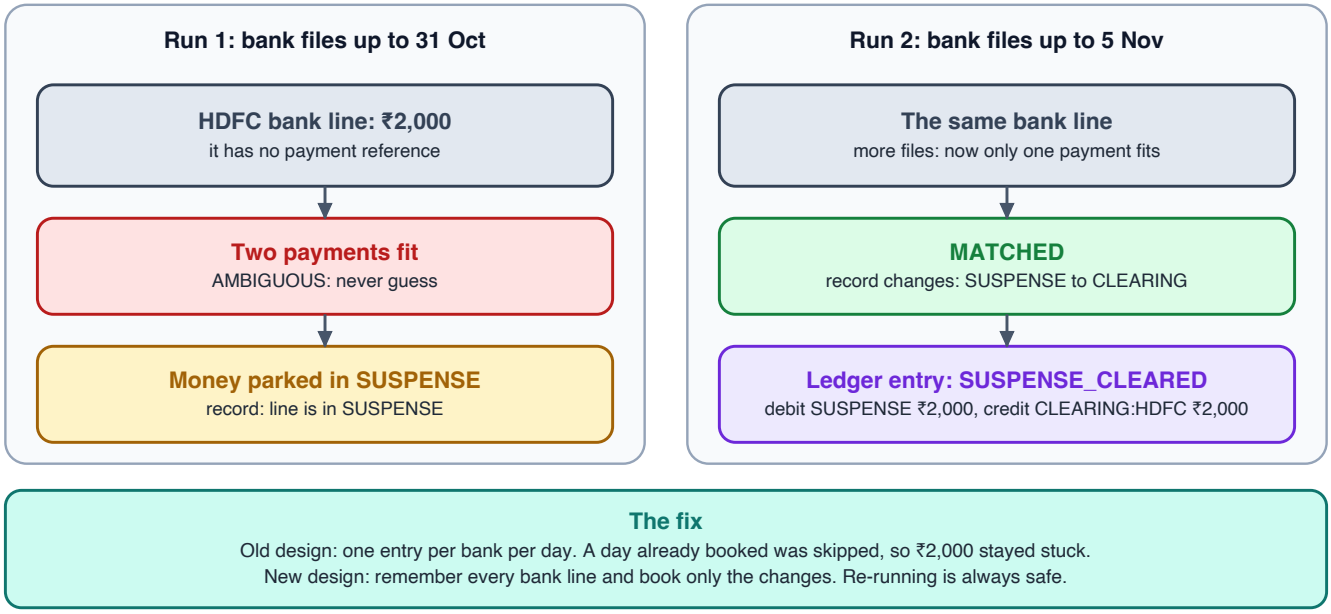


Figure 9. Why cash is tracked per bank line: money parked in suspense moves to clearing once it is identified.

The ledger entries and the per-line record are written in one transaction, and two audit checks prove they agree. Re-running a reconciliation is now **idempotent**: it is always safe and never books twice. In the 200k run, the second run printed: *"1 lines identified since the last run moved suspense → clearing (₹2,000.00)"*.

7.3 The tie-out: the accountant's proof

For each bank, two numbers are compared:

```
what we captured  −  cash the bank actually paid (booked to its clearing account)
                  =  open items the matching engine reported
                     (in transit + missing + in review + amounts short-paid)
```

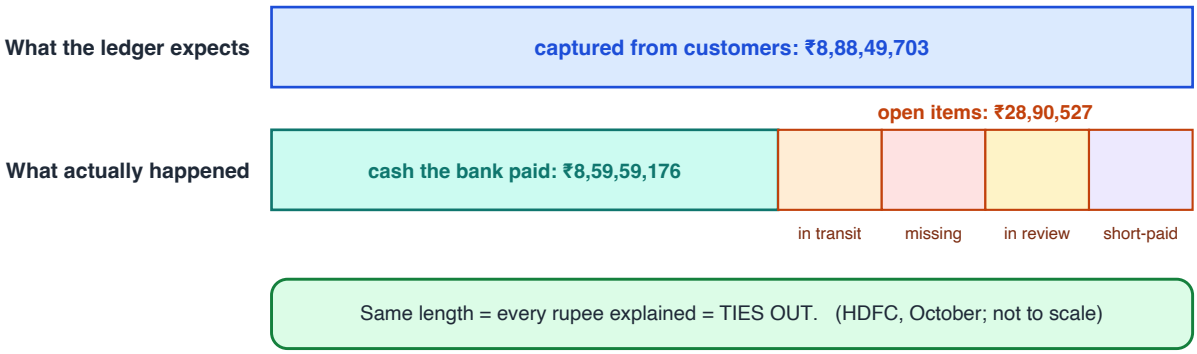


Figure 10. The tie-out for one bank: captured equals cash received plus the open items, to the paisa.

If both sides are equal, every rupee is explained. Accountants call this **tying out**. It connects three independent things: the ledger, the bank files and the MapReduce results. A bug in any one of them shows up as a difference.

```
Bank            Captured  Settled (cash booked)     Still owed     Open items  Result
HDFC     ₹8,88,49,703.01        ₹8,59,59,176.30  ₹28,90,526.71  ₹28,90,526.71  TIES OUT
ICICI    ₹7,21,83,667.16        ₹6,98,98,311.92  ₹22,85,355.24  ₹22,85,355.24  TIES OUT
CARDNET  ₹9,01,65,665.28        ₹8,40,30,894.21  ₹61,34,771.07  ₹61,34,771.07  TIES OUT
```

8. Part 6: Hive (the analytics warehouse)

A **data warehouse** is a database made for analysing history: large reads, few updates. Two kinds of database work are worth naming:

- **OLTP** (online transaction processing): many small, fast writes, like booking a payment. That is MySQL's job here.
- **OLAP** (online analytical processing): big questions over lots of data, like "how much did each bank settle this month?" That is Hive's job.

Keeping them apart means a heavy report can never slow down live payments.

Hive lets you run SQL (its version is **HiveQL**) over files. Some Hive ideas used here:

- **External table:** a Hive table over files that *we* own. Hive only reads them, and dropping the table never deletes the files. PayRecon has four: `ledger_payments`, `bank_lines`, `recon_matches` and `recon_exceptions` (`hive/01_create_tables.hql`).
- **Schema-on-read:** the files are plain text, and the column structure is applied when Hive *reads* them. A normal database checks structure when you write (**schema-on-write**).
- **Partition:** a sub-folder per value of a column, for example `recon_matches/run_id=3/` or `bank_lines/value_date=2026-10-12/`. A query filtered by `run_id = 3` reads only that folder. Skipping the rest is called **partition pruning**.
- `MSCK REPAIR TABLE … SYNC PARTITIONS` tells Hive's catalog (the **metastore**) which partition folders were added or removed. PayRecon runs it after every reconciliation.

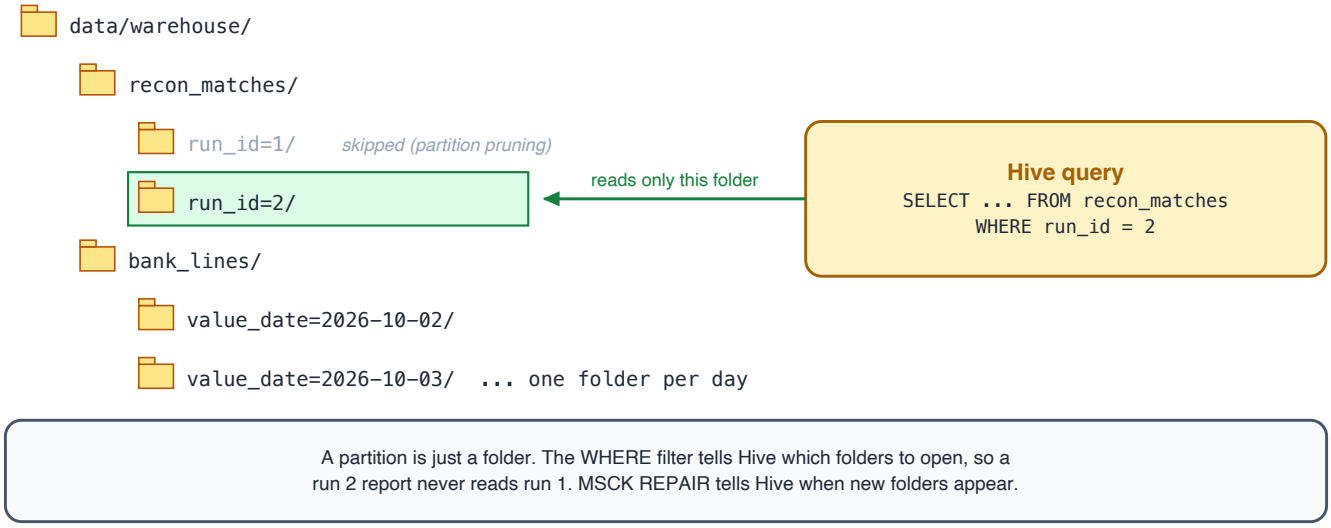


Figure 11. Hive partitions are folders. Filtering on the partition column reads only the folders it needs.

The five reports (`./payrecon report ...`):

Report	What it answers	SQL features
`close`	Does each bank tie out? How much is in transit, missing, in review, short-paid, in suspense?	CTEs (`WITH`), joins, `CASE`
`aging`	How old are the unsettled payments?	`DATEDIFF`, buckets
`breakdown`	Exceptions by type and bank; top 10 shops by money stuck	`RANK() OVER (...)`
`lag`	How many days each bank takes to settle; 95th percentile	`percentile_approx`
`daily`	Auto-match rate per day, and a running total	`SUM() OVER (ORDER BY … ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)`

A **window function** computes a value for each row using a "window" of related rows, without collapsing them into one row the way `GROUP BY` does. `RANK()` numbers rows by a sort order. A running total is `SUM` over "all rows up to this one". A **percentile** is the value below which a given share of rows fall: the 95th percentile settlement lag means 95% of payments settled at least that fast.

Hive's close report and the Java tie-out are computed independently and agree.

9. Part 7: MongoDB (documents)

A **document database** stores **documents**: JSON-like records that can contain lists and nested objects. Documents live in **collections** (like tables, but each document can have a different shape). MongoDB stores them as **BSON**, a binary form of JSON.

`bank_raw_records` holds every bank line exactly as received, with the bank's *own* field names, plus a normalised view:

```
{
  _id: "CARDNET:cardnet_settlement_2026-10-06.jsonl:193",
  bank: "CARDNET", file: "...", line_no: 193,
  raw: "<the original line>",
  parse_status: "OK",
  fields: { settlement_id: "stl_2755758982", txn: { merchant_ref: "pay_9207263263", auth_code: "360612" },
            gross: { value: "10358.24", currency: "INR" }, fees: { interchange: "0.00", scheme: "0.00" } },
  normalized: { direction: "CREDIT", reference: "pay_9207263263", ref_quality: "FULL",
                amount_minor: 1035824, value_date: "2026-10-06" }
}
```

Why a document store? HDFC lines have 6 CSV columns, ICICI lines have 6 different pipe fields, and CARDNET lines are nested JSON. A new bank tomorrow would have yet another shape. Documents keep each one as-is, with no table change needed. When an analyst investigates an exception, `./payrecon exceptions show <id>` displays this original record.

- **Indexes** (a sorted lookup structure, like a book's index) on `normalized.reference`, on `(bank, normalized.value_date)` and on `parse_status` make those lookups fast.
- Writes use **upserts** (update the document if it exists, insert it if not) in **bulk writes** (thousands of operations per round trip), so re-running never creates duplicates.

`payout_statements` holds one document per payout, containing the shop, the batch, the bank status from job 3, a summary, and the full list of payments and refunds inside it. A shop's statement is **one read** instead of joining four ledger tables.

This pattern is called **CQRS** (command query responsibility segregation): the *write model* (the normalised ledger in MySQL) is separate from a *read model* (copies shaped for reading, rebuilt from the write model).

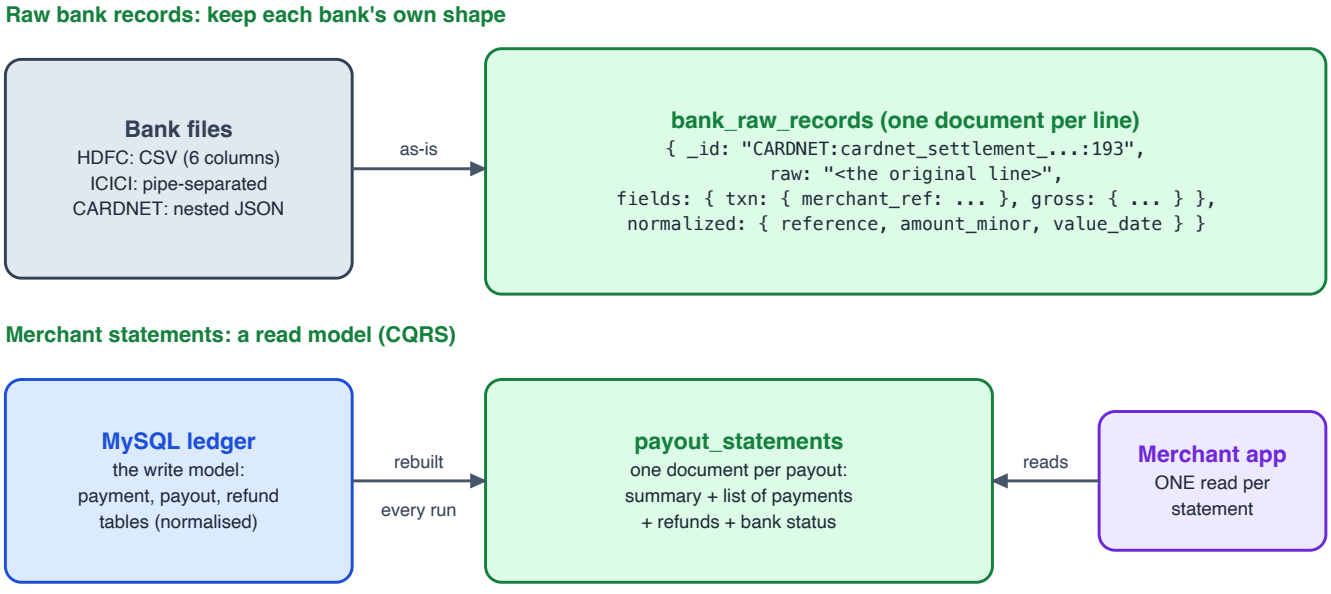


Figure 12. MongoDB's two jobs: raw records exactly as the banks sent them, and ready-made statements.

Honestly, MongoDB has the lightest role of the four tools. At this size, MySQL with JSON columns could also do it.

10. Part 8: the exceptions workflow (MySQL)

Analysts work through exceptions like a **work queue**: each one takes the next item, investigates it, and resolves it.

- `claim` uses `SELECT … FOR UPDATE SKIP LOCKED`. It locks the next open item, and **skips** rows another analyst has already locked instead of waiting for them. Ten analysts can claim at the same time and never get the same item. The stress test proves it with 6.
  - Subtle detail: this only works because `ORDER BY exception_id` follows an index. If MySQL had to sort first, it would read and lock *every* matching row, and the others would find nothing to skip to. That is why exceptions are inserted in priority order: id order *is* priority order.
- `resolve` uses **optimistic locking**. Each row has a `version` number, and the update says `... WHERE exception_id = ? AND version = ?`. If someone changed the row in between, zero rows are updated and we refuse. Optimistic means "don't lock, but detect conflicts". Pessimistic means "lock first", like the refunds in section 4.3.
- Every claim, resolve and release is written to `exception_event`, the **audit trail**: who did what, and when.

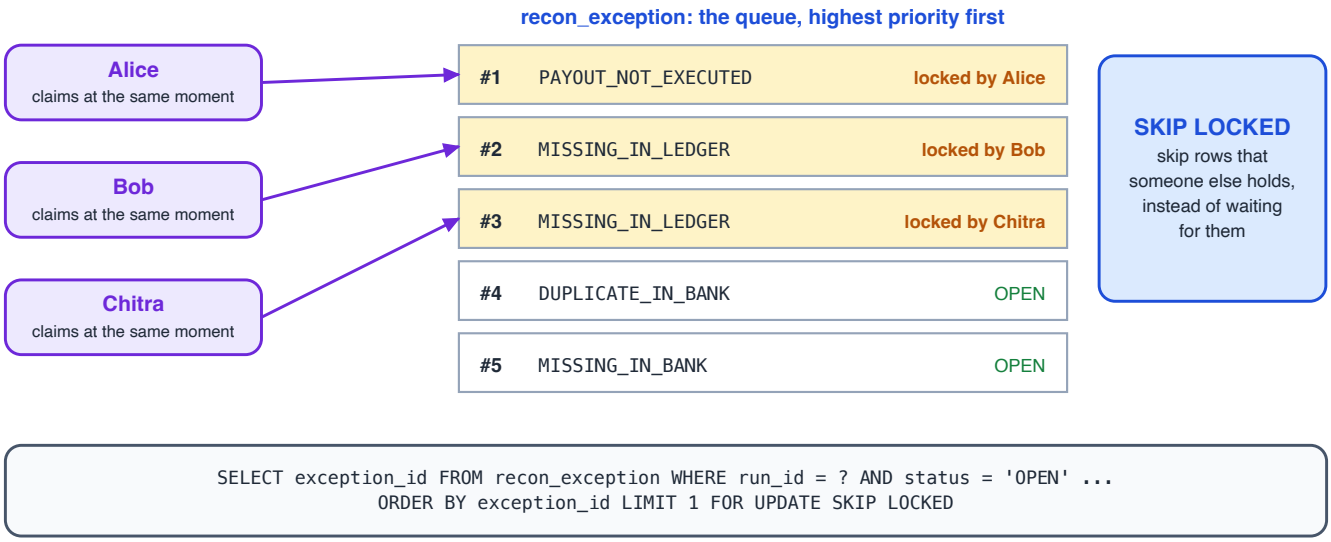


Figure 13. Three analysts claim at the same moment and each gets a different item. Nobody waits.

11. Part 9: how we know it is correct

There are five independent layers:

1. **Unit tests:** 27 of them (`mvn test`). They cover money maths, all three parsers, the three matchers (written as pure functions with no Hadoop or database, so they are easy to test) and the ledger rules.
2. `verify`**:** every planted problem must be reported, and nothing else.
3. **Tie-out:** each bank's numbers must balance against the open items.
4. `audit`**:** 9 accounting rules over the whole ledger.
5. `stress`**:** correctness under real concurrency.

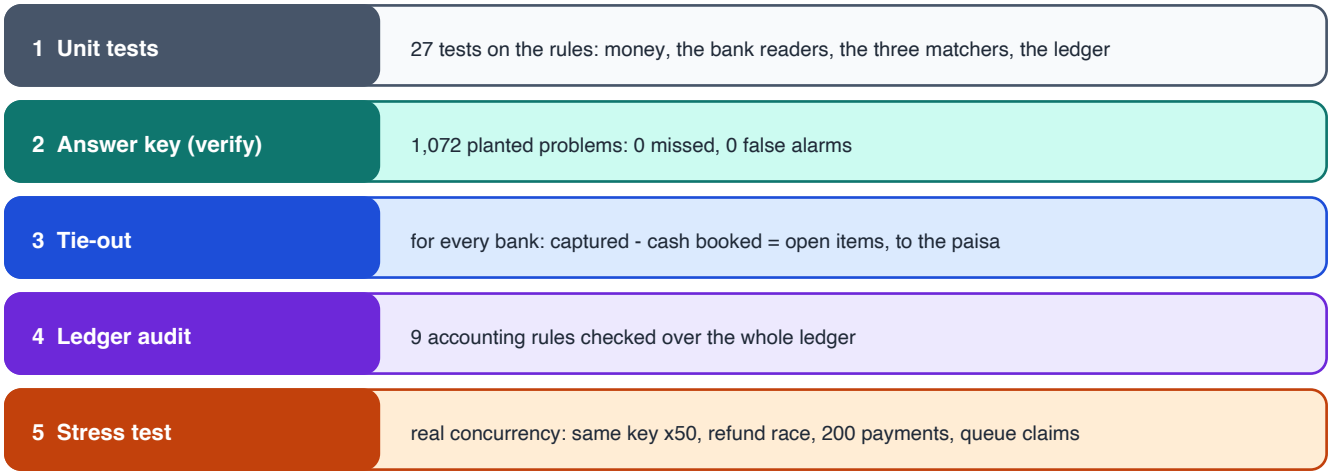


Figure 14. Five independent checks. Each one catches a different kind of mistake.

Building with these checks found three real bugs:

- the wrong copy of a duplicate was chosen (section 6.2)
- money was stuck in suspense (section 7.2)
- rupee amounts printed with Western instead of Indian digit grouping

12. Results (October 2026: 1,000 shops, 200,000 payments, MacBook, Docker with 4 GB)

What	Result
Month generated (₹25.1 crore of payments, 3,515 refunds, 9,269 payouts in 60 batches)	13.0 s
Bank files	105 files, 200,186 lines
Reconciliation run 1 (cutoff 31 Oct), all six steps	21.9 s (MapReduce: 7.7 s)
Matched in job 1 by exact reference	191,321
Matched in job 2 by amount + date window	515
Matched in job 3 (payout batches)	58 of 60
Auto-matched, of payments due by the cutoff	99.83% (7,837 more still in transit)
Run 2 (cutoff 5 Nov): auto-matched	99.92%, in 26.6 s
`verify`	PASS: 0 missed, 0 false alarms
Tie-out	every bank TIES OUT, in both runs

13. Design decisions, trade-offs and limits

- **Matching rules are pure functions** (`recon/match`), separate from the Hadoop classes (`recon/mr`). They can be tested in milliseconds, and the same rules could run on Spark tomorrow.
- **There is one parser** for all bank formats, used by MapReduce and by MongoDB.
- **Ambiguity is never guessed.** A wrong automatic match is worse than a manual review, because it hides money.
- **The clock starts before reading any input**, so the reported runtime is honest.

- **Re-runs are idempotent**, both for the API (idempotency keys) and for reconciliation (the per-line cash record).

Limits (what a real company would add):

- **Spark or a real Hadoop cluster** instead of local mode.
- **A connection pool** such as HikariCP instead of one new MySQL connection per request.
- **Sharded "hot accounts".** Every payment updates `FEE_REVENUE` and a clearing account, so those rows become a bottleneck under heavy load.
- **Multi-currency and FX.** The code assumes INR.
- **Payments from earlier months** settling in this month.
- **Kafka** for real-time events instead of daily batch files.
- **Authentication and PCI-DSS** (the security standard for card data).

14. Where things are in the code

Folder / file	What it does
`Main.java`, `cli/Args.java`	the `./payrecon` commands
`config/Config.java`	reads `.env`; passwords are never in code
`db/Db.java`	transactions with retry on deadlock
`ledger/LedgerService.java`	journal entries, payments, refunds, lock order
`ledger/LedgerAudit.java`	the 9 checks
`api/ApiServer.java`	REST API and idempotency
`api/StressTest.java`	concurrency scenarios
`bank/BankParsers.java`	the one parser
`gen/DataGenerator.java`, `gen/BankFileWriter.java`	month of data, bank files and answer key
`recon/match/*Matcher.java`	the matching rules (pure, unit-tested)
`recon/mr/*`	the three MapReduce jobs and the secondary-sort classes
`recon/ReconOrchestrator.java`	the six steps of a run
`recon/CashApplication.java`, `recon/TieOut.java`	ledger booking and the proof
`recon/Verifier.java`	comparison with the answer key
`exceptions/ExceptionWorkflow.java`	the analysts' queue
`mongo/MongoStore.java`, `recon/StatementBuilder.java`	MongoDB
`hive/HiveRunner.java`, `hive/*.hql`	Hive
`src/main/resources/sql/mysql_schema.sql`	the 15 MySQL tables

Glossary

- **Accounting equation:** assets = liabilities + revenue − expenses. It always holds in a correct ledger.
- **ACID:** the four promises of a database transaction: atomic, consistent, isolated, durable.
- **Acquiring bank:** the bank that processes payments for a merchant or aggregator, and settles the money.
- **Actionable exception:** a difference that needs a person to look at it.
- **API:** a way for programs to talk to a system, here over HTTP.

- **Asset / liability / revenue:** what we own or are owed; what we owe; what we earned.
- **Associative:** (a+b)+c = a+(b+c). It is what makes combiners safe.
- **Audit trail:** a permanent record of who did what, and when.
- **Basis point (bps):** 0.01%. 200 bps = 2%.
- **BSON:** the binary form of JSON that MongoDB stores.
- **Cash application:** booking money actually received into the right ledger account.
- **Chart of accounts:** the list of all accounts in a ledger.
- **Clearing account:** money a bank owes us that has not arrived yet.
- **Collection:** a group of MongoDB documents, similar to a table.
- **Combiner:** a mini-reducer that runs on the map side to shrink data before the shuffle.
- **Composite key:** a key made of several parts.
- **Constraint:** a rule the database enforces itself (CHECK, unique key, foreign key).
- **Counter (Hadoop):** a global tally that every task can increment.
- **CQRS:** keeping separate models for writing (the ledger) and reading (statements).
- **DAG:** steps connected by dependency arrows, with no loops.
- **Data warehouse:** a database made for analysing historical data.
- **Deadlock:** two transactions each waiting for a lock the other holds.
- **Debit / credit:** the two sides of every double-entry posting.
- **Document (database):** a JSON-like record that can contain lists and nested objects.
- **Double-entry bookkeeping:** every money movement is recorded on two sides that must balance.
- **Endpoint:** one URL of an API, such as `POST /v1/payments`.
- **External table:** a Hive table over files that Hive does not own.
- **Foreign key:** a column that must match a row in another table.
- **Grouping comparator:** decides which records share one `reduce()` call.
- **Ground truth / answer key:** the known correct answer, used for testing.
- **Hadoop / MapReduce:** a big-data system, and its map → shuffle → reduce model.
- **Hive / HiveQL:** SQL over files, and its SQL dialect.
- **HTTP status code:** a number in every HTTP reply (201 created, 409 conflict, ...).
- **Idempotent / idempotency key:** safe to repeat; the client-chosen id that makes retries safe.
- **Index:** a sorted lookup structure that makes searches fast.
- **In transit:** not settled yet, but still within normal settlement time.
- **Isolation level:** how much transactions running at the same time can see of each other.
- **Journal entry / posting:** one business event; one line (account, debit or credit, amount) of it.
- **Key-value pair:** the unit of data in MapReduce.
- **Ledger:** the official record of all money movements.
- **Local mode:** running Hadoop inside one program, with no cluster.
- **Metastore:** Hive's catalog of tables and partitions.
- **MultipleOutputs:** writing several named outputs from one MapReduce job.
- **Normal side:** the side (debit or credit) on which an account's balance grows.
- **OLTP / OLAP:** many small live transactions, versus big analytical questions.
- **Optimistic / pessimistic locking:** detecting conflicts with a version number, versus locking before changing.
- **Partition (Hive):** a sub-folder per column value; lets queries skip data (partition pruning).
- **Partitioner (MapReduce):** decides which reducer receives a record.

- **Payment aggregator:** a company that accepts payments for many merchants.
- **Payout / payout batch:** sending a merchant their money; all payouts to one bank on one day, sent as one debit.
- **Percentile:** the value below which a given share of the data falls.
- **Primary key / unique key:** a column (or columns) whose values can never repeat.
- **Race condition:** a bug that depends on the timing of things running at once.
- **Read model:** a copy of data shaped for fast reading.
- **Reconciliation:** comparing our records with the bank's and explaining every difference.
- **Reduce-side join:** joining two sources by sending both to the reducer under the same key.
- **REST:** a style of API built on HTTP methods and URLs.
- **Row lock:** a lock on a single database row.
- **Schema-on-read / schema-on-write:** structure applied when reading files, versus checked when writing rows.
- **Secondary sort:** getting each reducer group's values in sorted order, using a composite key, a partitioner and a grouping comparator.
- **Seed:** the starting number of a random generator; the same seed gives the same data.
- **Settlement / settlement file:** the bank transferring collected money; the bank's list of what it transferred.
- **SKIP LOCKED:** skip rows locked by others instead of waiting; used for work queues.
- **Split:** a piece of an input file given to one mapper.
- **Suspense account:** a parking place for money not yet identified.
- **Tie-out:** proving two sets of numbers explain each other exactly.
- **Transaction:** a group of database changes that all happen or none do.
- **Trial balance:** checking that total debits equal total credits.
- **Two-pointer sliding window:** walking two sorted lists together so each item is looked at only a few times.
- **Upsert:** update if it exists, insert if not.
- **UTR / RRN:** banks' own transaction reference numbers.
- **Value date:** the day the bank actually moved the money.
- **Window function:** SQL that computes a value from related rows without grouping them away.